DREW

Lecture 34: Pointers and Arrays

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

11/22/2024

Pointers in C

- A pointer is an address. In C, a pointer is also a variable.
- To **declare** a pointer: We use * to make a variable a pointer.

  dataType varName or dataType *varName*
- A NULL pointer, *int*p = NULL;*.
- Two operations on pointer:

1. **p*: returns the value pointer *p* is pointing towards. **Dereferencing**
2. *&var*: returns the address of variable *var*.
   - **Recall**: scanf("%d", &num); scanf("%c", &ch);

DREW

Pointers in C

- **Types of Dereferencing**
  - *p*: returns the value pointer *p* is pointing towards.
  - *0x3000*: returns the value at memory location 0x3000.

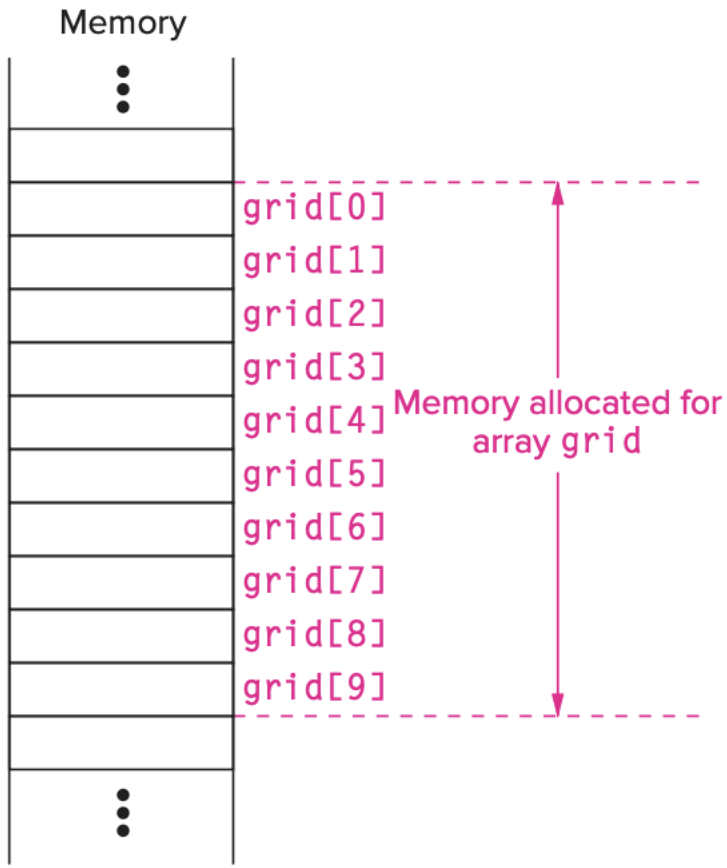
```
int obj = 4;
int *ptr1 = &obj;
Int **ptr2 = &ptr1;
```

  - **ptr2*: returns the value pointer *ptr2* is pointing towards; i.e., *ptr1*.
  - ***ptr2*: returns the value pointer *ptr1* is pointing towards.

Arrays in C

- Earlier, defined array as
  - *dataType varName[] = init.* OR
  - *dataType varName[num_entries]*
- Accessed through index
  - *varName[index];*
- **Ex.**: define datatype *grid[10]*

grid[0] *grid[9]*

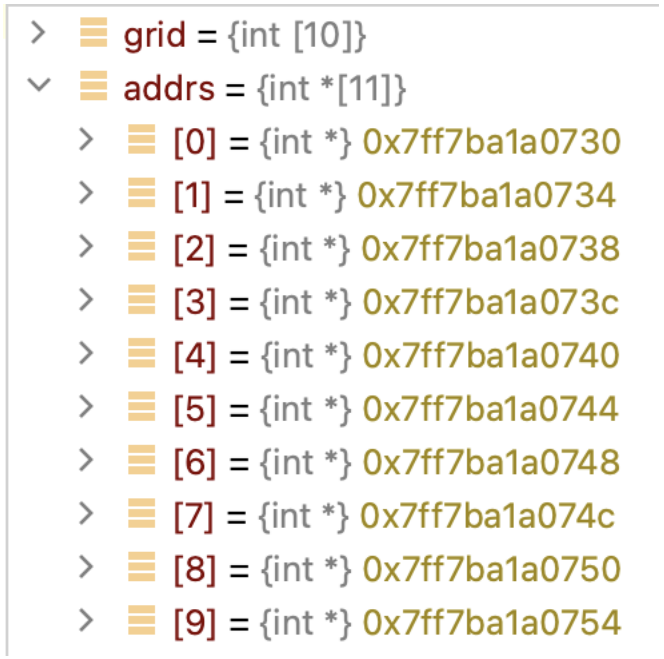


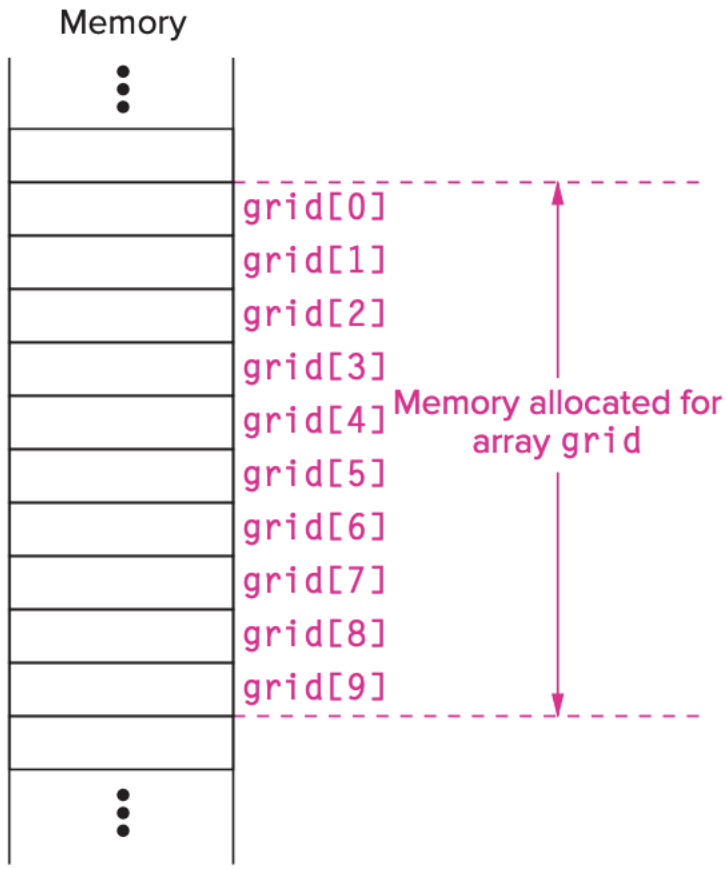
Arrays in C

- Now, since we know how pointers work, can write a simple program to check this.
- In fact, we are using a pointer array.

```
int grid[10];
int* addrs[10];

for (int i = 0; i < 10; i++)
        addrs[i] = &grid[i];
```





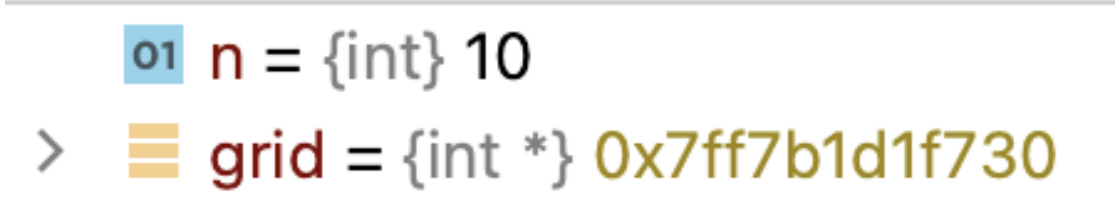
DREW

Passing Arrays – as Arguments

- We can pass arrays as arguments to functions, just as any other datatypes.

- However, what's being passed in are fundamentally different.
  - When passing numbers and chars, only the value is passed in.
    (Passing by value)
  - When passing an array, the address of the 0th. index is passed in.
    (Passing by reference) [Otherwise, need to copy all entries.]

```
void test_func(int grid[], int n) { … }
In main():
        int grid[10];
        int n = 10;
        test_func(grid, n);
```



Passing Arrays – as Arguments

- Whereas we CANNOT return arrays back to caller functions!
- Again, otherwise, will need to copy back all the entries. NOT good!

- The solution is similar to how to pass arrays as arguments.
  (Passing by reference instead)

- **Ex.**: Pass an array with integers to a function that cubes the entries.

Passing Arrays – as Returns

- Whereas we CANNOT return arrays back to caller functions!
- Again, otherwise, will need to copy back all the entries. NOT good!

- The solution is, again, Passing by reference.
- **Ex.**: Pass an array with integers to a function that cubes the entries.
- In *main()*:

```
int nums[] = {1,2,3,4,5,6};
int len = (int) sizeof(nums)/sizeof(int);
int* cubed = cubic(nums, len);


for (int i = 0; i < len; i++)
        printf("%d ", cubed[i]);
```

Passing Arrays – as Returns

```
int* cubic (int nums[], int len) {

//   for (int i = 0; i < len; i++) {
//       nums[i] = nums[i]*nums[i]*nums[i];
//   }

    int new_nums[len];
    for (int i = 0; i < len; i++)
        new_nums[i] = nums[i]*nums[i]*nums[i];

//   return &new_nums[0];
    return new_nums;
}
```

// Changes entries of nums in the memory. Return type is void then.

// Although nums here is addr. of oth. index, still access it as an array.

// Lastly, still returning an addr, that of the oth. index, Or simply, the array name.

DREW

Pointers and Arrays

- Turns out, an array name is essentially a pointer of its 0th. index.

- **Ex.:**

```
char word[10] = "HelloWorld";
char* cptr;
cptr = word;          // cptr points to word[0]
```

- Hence, these are the same:

```
printf("%c", *cptr);
printf("%c\n", *(cptr+1));
```

and

```
printf("%c", *word);
printf("%c\n", *(word+1));
```

and

```
printf("%c", word[0]);    printf("%c\n", word[1]);
```

DREW

Pointers and Arrays

- Below, each row contains 3 equivalent forms:

Table 16.1 Expressions with Similar Meanings

	Using a Pointer	Using Name of Array	Using Array Notation
Address of array	`cptr`	`word`	`&word[0]`
0th element	`*cptr`	`*word`	`word[0]`
Address of element n	`(cptr + n)`	`(word + n)`	`&word[n]`
Element n	`*(cptr + n)`	`*(word + n)`	`word[n]`

Here we assume `cptr` is a `char *` and `word` is a `char` array and `cptr = &word[0];`

Pointers and Arrays

- Since Pointers and Arrays are essentially the same in C.
- How do we define a block of space using pointer?

  (i.e., equi. of *char word[10]*)
- (*.BLKW* in LC-3) *malloc()* in C
- Since it's a pointer, we want it to be the address of the first entry.

- Thus, *char* myArray = (char*) malloc(sizeof(char)*LEN);*

- Allocate in the memory space for value of LEN this many chars.
- This is multiplication, not pointer.
- Since it's a pointer, let's cast it as one pointing towards chars.

Pointers and Arrays

- Thus, *char* myArray = (char*) malloc(sizeof(char)*LEN);*

- **Ex.:**

```
char* myArray = (char*) malloc(sizeof(char)*30);
scanf("%s", myArray);
printf("%s\n", myArray);

free(myArray);
```

// Define a char array
of length 30 using pointer.

// Since manually allocated memory
to this array, need to release that block
after done.
Otherwise, may run out of memory!

Passing Arrays – as Returns

- Now, back to the cube example:

```
int* cubic (int nums[], int len) {

    int* new_nums = (int*) malloc(sizeof(int)*len);
    for (int i = 0; i < len; i++)
        new_nums[i] = nums[i]*nums[i]*nums[i];

//    return &new_nums[0];
    return new_nums;
}
```

// Although new_nums is a pointer, still access it as if it's an array.

// Lastly, still returning an addr.

Passing Arrays – as Returns

- In *main()*:

```
int nums[] = {1,2,3,4,5,6};
int len = (int) sizeof(nums)/sizeof(int);
int* cubed = cubic(nums, len);

for (int i = 0; i < len; i++)
        printf("%d ", cubed[i]);

free(cubed);        // This is the same pointer as new_nums
                    declared in cubic(), so free this is the same as
                    free the malloc we made.
```

Multidimensional Arrays

- Recall: Saw double arrays when dealing with matrices without passing.

- Now, we know how to deal with pointers and arrays, can translate to double-pointers and they are essentially 2-D arrays.
- Etc.

Thanksgiving break Schedule

- Still have lecture on 11/25 (M) (sorry 😭, but)
  - EXTRA CREDITS!! 2% towards total grade
  - Will talk about project
- HW7 due on 11/26 (T)
- Happy Thanksgiving!





DREW